

Midterm Project

Real-Time Data Analysis and Decision Making

AITU Classroom Environmental & Energy Monitoring

Galymzhan Tamirlan Galymzhanuly
Student ID: 240542 | Group: BDA-2407
Astana IT University
Instructor: Serek Azamat

GitHub Repository: https://github.com/mtake-da/Midterm_RTDADM.git

October 2026

Contents

1	Introduction	2
2	Data Scenario and Stream Architecture	2
2.1	The Room and Sensor Setup	2
2.2	Data Schema	2
2.3	Lecture Simulation Phases	2
3	Implementation	3
3.1	Stream Generator (<code>generate_stream</code>)	3
3.2	Stream Processor Architecture (<code>StreamProcessor</code>)	3
3.2.1	Step 1: Online Imputation	3
3.2.2	Step 2: Real-Time Indicator Engineering	4
3.2.3	Step 3: Multi-Tier Rule-Based Anomaly Detection	4
3.2.4	Step 4: Stream Ingestion and Event Structuring	5
4	Results	5
4.1	Summary Statistics	5
4.2	Incident and Anomaly Breakdown	5
4.3	Sample Processed Window: Ventilation Breakdown	5
4.4	Monitoring Dashboard Visualization	6
5	Discussion	6
5.1	System Strengths and Observations	6
5.2	Limitations and Future Work	7
6	Conclusion	7

1 Introduction

For this midterm project in Real-Time Data Analysis and Decision Making (RTDADM), I built a real-time stream processing and monitoring system for an academic classroom at Astana IT University (Room C1.3.101). The system builds upon the classroom IoT sensor dataset from Assignment 2, scaling and streaming the observations over an extended 60-minute lecture session.

The primary objective is to ingest sensor packets sequentially one minute at a time, apply online imputation for missing sensor telemetry without look-ahead bias, compute rolling statistics and physical domain indicators via sliding buffers, and detect multi-tier environmental and operational anomalies in real time.

2 Data Scenario and Stream Architecture

2.1 The Room and Sensor Setup

Room C1.3.101 is a lecture and lab space at AITU with a nominal maximum capacity of 40 students. The room is instrumented with four physical sensors:

- **Temperature Sensor** (Temp_C): Measures indoor ambient air temperature in °C.
- **CO₂ Sensor** (CO2_ppm): Measures indoor carbon dioxide concentration in parts per million (ppm).
- **Occupancy Camera** (Occupancy): Provides computer-vision headcount counts of people present.
- **Smart Power Meter** (Power_kW): Measures aggregate electrical demand of lighting, HVAC terminals, and workstation rigs in kilowatts (kW).

Incoming sensor packets arrive at 1-minute intervals. The stream simulation uses a fixed pseudo-random seed (42) to guarantee strict determinism and reproducibility.

2.2 Data Schema

The schema for incoming packets yielded by the stream generator is detailed in Table 1.

Table 1: Incoming sensor packet schema

Field	Type	Format / Units	Normal Operating Range
Time	string	YYYY-MM-DD HH:MM:00	10:00:00 – 10:59:00
Room	string	Categorical	C1.3.101
Temp_C	float	°C	20.0 – 24.0
CO2_ppm	float	ppm	400 – 1000
Occupancy	integer	persons	0 – 40
Power_kW	float	kW	1.5 – 7.0

2.3 Lecture Simulation Phases

The 60-minute timeline is divided into six operational lecture phases:

1. **10:00–10:11 (Assignment 2 Baseline):** 12 historical records containing 3 intentional sensor dropouts at minutes 10:02, 10:04, and 10:07.
2. **10:12–10:25 (Full Lecture Capacity):** Class reaches full occupancy (35–40 students); steady environmental conditions.
3. **10:26–10:35 (Ventilation Failure):** Rapid CO₂ buildup, crossing the 1000 ppm warning threshold and surging into critical levels (> 1200 ppm).
4. **10:36–10:45 (Thermal Accumulation):** Ambient heat accumulates steadily under full attendance, exceeding the 24.0°C comfort boundary.
5. **10:46–10:50 (Laboratory Practical & Power Surge):** Intensive student computing workstations activate, pushing electrical load above the 6.0 kW threshold.
6. **10:51–10:59 (Classroom Dismissal):** Students depart ($|\Delta\text{Occupancy}| \geq 6$), ventilation recovers, and power returns to idle.

3 Implementation

3.1 Stream Generator (generate_stream)

The `generate_stream(total_records=60, seed=42)` function initializes the 12 empirical records from Assignment 2 and iteratively simulates the remaining 48 minutes according to the lecture timeline. Gaussian noise ($\mathcal{N}(\mu, \sigma)$) models physical variation and sensor jitter. In addition to the 3 baseline dropouts (10:02, 10:04, 10:07), three realistic transmission dropouts are injected into the extended stream:

- **10:18:** Temp_C is missing (None)
- **10:33:** CO2_ppm is missing (None)
- **10:44:** Occupancy is missing (None)

3.2 Stream Processor Architecture (StreamProcessor)

The streaming core is encapsulated in the `StreamProcessor` class. Records are ingested one by one via `process(record)` with zero look-ahead capability, maintaining state in sliding ring buffers (`collections.deque(maxlen=5)`):

3.2.1 Step 1: Online Imputation

Missing sensor fields are imputed dynamically prior to indicator calculation:

- **Continuous Sensors (Temp_C, CO2_ppm):** When at least two previous valid values exist in the buffer (x_{t-2}, x_{t-1}), the missing value is predicted using linear trend extrapolation:

$$\hat{x}_t = \frac{x_{t-2} + x_{t-1}}{2} + (x_{t-1} - x_{t-2}) = x_{t-1} + \frac{x_{t-1} - x_{t-2}}{2} \quad (1)$$

If insufficient history is present in the buffer, the engine falls back to the last known state. For instance, at 10:02 the imputed temperature of 21.80°C smoothly bridges

21.5°C and 21.7°C, and at 10:04 the imputed CO₂ concentration of 735.0 ppm smoothly extrapolates the observed trend.

- **Discrete Sensors (Occupancy):** Forward-fills the most recent valid headcount.

Whenever an imputation occurs, the event is immediately flagged with a `SENSOR_DROPOUT` alert.

3.2.2 Step 2: Real-Time Indicator Engineering

Using sliding buffers of length $W = 5$, the engine derives multiple rolling metrics:

1. **Rolling Moving Averages:** Dampens minute-by-minute measurement noise:

$$\bar{T}_t = \frac{1}{N} \sum_{i=0}^{N-1} T_{t-i}, \quad \bar{C}_t = \frac{1}{N} \sum_{i=0}^{N-1} C_{t-i}, \quad \bar{P}_t = \frac{1}{N} \sum_{i=0}^{N-1} P_{t-i} \quad (N \leq 5) \quad (2)$$

2. **Rolling Volatility (Standard Deviation):** Tracks volatility in temperature and electrical demand:

$$\sigma_T^{(t)} = \sqrt{\frac{1}{N} \sum_{i=0}^{N-1} (T_{t-i} - \bar{T}_t)^2}, \quad \sigma_P^{(t)} = \sqrt{\frac{1}{N} \sum_{i=0}^{N-1} (P_{t-i} - \bar{P}_t)^2} \quad (3)$$

3. **Temperature Velocity (Delta):** Measures sudden thermal shift: $\Delta T_t = T_t - T_{t-1}$.

4. **Domain Composite Indicators:**

- **Occupancy Density:** $d_t = \text{Occupancy}_t / 40.0$ (proportion of nominal room capacity).
- **Specific Energy per Occupant:** $e_t = P_t / \max(\text{Occupancy}_t, 1)$ (kW per student).
- **Comfort Compliance Boolean:** $\text{comfort_ok}_t \iff (20.0 \leq T_t \leq 24.0) \wedge (C_t \leq 1000.0)$.

3.2.3 Step 3: Multi-Tier Rule-Based Anomaly Detection

Each incoming packet is evaluated against operational thresholds defined in Table 2.

Table 2: Anomaly detection rules and severity tiers

Alert Identifier	Severity Tier	Trigger Condition
SENSOR_DROPOUT	Data Quality Fault	Any sensor field is missing (None)
CO2_HIGH	Comfort Warning	CO ₂ > 1000 ppm and ≤ 1200 ppm
CO2_CRITICAL	Safety Alarm	CO ₂ > 1200 ppm
OVERHEATING	Comfort Warning	Temperature > 24.0°C
UNDERHEATING	Comfort Warning	Temperature < 20.0°C
POWER_SURGE	Hazard Alarm	Power > 6.0 kW
RAPID_OCC_CHANGE	Dynamic Shock	$ \Delta \text{Occupancy} \geq 6$ persons/min

3.2.4 Step 4: Stream Ingestion and Event Structuring

Each processed record yields an enriched record dictionary (16 attributes) appended to `proc.history`, and every triggered alert appends an incident entry to `proc.anomalies`. Converted into pandas DataFrames (`df` and `df_anom`), these structures provide full auditability and feed the visualization suite without any batch-mode lookahead bias.

4 Results

4.1 Summary Statistics

Sequential execution over all 60 minutes produced the summary statistics shown in Table 3.

Table 3: Statistical summary of processed stream variables ($N = 60$)

Metric	Temp (°C)	CO ₂ (ppm)	Power (kW)	Occupancy (persons)
Mean	23.93	1053.23	5.50	33.95
Std	0.93	190.99	1.49	9.85
Min	21.50	620.00	1.50	0.00
25%	23.58	952.92	4.59	30.50
50%	23.98	1010.50	5.90	39.50
75%	24.74	1258.75	6.57	40.00
Max	25.07	1324.00	7.10	40.00

4.2 Incident and Anomaly Breakdown

A total of **96 anomaly events** were logged across the 60-minute duration:

- **OVERHEATING (30 events):** Consecutive from 10:30 through 10:59 as indoor temperature hovered between 24.03°C and 25.07°C.
- **POWER_SURGE (28 events):** Occurred intermittently during high activity (10:17–10:18) and continuously during lab tasks (10:25–10:50), peaking at 7.10 kW.
- **CO2_CRITICAL (20 events):** Sustained uninterrupted exposure from 10:32 to 10:51, with concentrations reaching 1324.0 ppm.
- **CO2_HIGH (10 events):** Recorded during the buildup phase (10:26–10:31) and the clearing phase after lecture end (10:52–10:55).
- **SENSOR_DROPOUT (6 events):** Correctly detected and imputed at 10:02, 10:04, 10:07, 10:18, 10:33, and 10:44.
- **RAPID_OCC_CHANGE (2 events):** Captured during rapid classroom exit at 10:53 (−6 students) and 10:55 (−6 students).

4.3 Sample Processed Window: Ventilation Breakdown

Table 4 illustrates the streaming indicators and concurrent alerts during the critical ventilation breakdown window (10:25–10:35), demonstrating how alerts transition in real time

from CO2_HIGH to compounding CO2_CRITICAL, OVERHEATING, and POWER_SURGE.

Table 4: Sample processed telemetry during ventilation incident (10:25–10:35)

Time	Temp_C	CO2_ppm	Pwr_kW	Roll CO ₂	Comfort	Triggered Alerts
10:25	23.77	993.0	6.01	968.4	True	POWER_SURGE
10:26	23.82	1023.2	6.36	982.7	False	CO2_HIGH; POWER_SURGE
10:27	23.85	1052.4	6.51	997.5	False	CO2_HIGH; POWER_SURGE
10:28	23.91	1073.8	6.47	1022.9	False	CO2_HIGH; POWER_SURGE
10:29	23.94	1101.8	6.49	1048.8	False	CO2_HIGH; POWER_SURGE
10:30	24.03	1139.4	6.40	1078.1	False	CO2_HIGH; OVERHEATING; POWER_SURGE
10:31	24.07	1173.4	6.51	1108.2	False	CO2_HIGH; OVERHEATING; POWER_SURGE
10:32	24.10	1204.2	6.38	1138.5	False	CO2_CRITICAL; OVERHEATING; POWER_SURGE
10:33	24.12	1219.6	6.53	1167.7	False	SENSOR_DROPOUT; CO2_CRIT; OVERHEAT; SURGE
10:34	24.16	1279.1	6.47	1203.1	False	CO2_CRITICAL; OVERHEATING; POWER_SURGE
10:35	24.19	1313.3	6.54	1245.9	False	CO2_CRITICAL; OVERHEATING; POWER_SURGE

4.4 Monitoring Dashboard Visualization

The 3-panel dashboard generated by the system is displayed in Figure 1:

1. **Panel 1 (Thermal & Air Quality over Time):** Dual y-axes displaying raw temperature readings against 5-minute rolling averages and statutory upper limits (24.0°C) on the left, paired with raw and rolling CO₂ concentration against warning (1000 ppm) and critical (1200 ppm) thresholds on the right.
2. **Panel 2 (Power Consumption & Headcount):** Shaded electrical demand curve with the 6.0 kW surge limit line on the left, paired with student room occupancy (0–40 persons) on the right.
3. **Panel 3 (Anomaly Event Timeline):** Chronological categorical scatter timeline mapping the occurrence of all 7 alert types across the 60 minutes with dedicated color encoding.

5 Discussion

5.1 System Strengths and Observations

Zero Lookahead Integrity: Unlike batch analytics where global imputation and global windowing can accidentally introduce future leakage, the stream processing engine operates strictly on current and historical records.

Smooth Imputation via Buffer Trend Extrapolation: When the temperature sensor dropped out at 10:02, the trend extrapolation $\hat{T}_{10:02} = 21.80^\circ\text{C}$ seamlessly continued the rising gradient ($21.5^\circ\text{C} \rightarrow 21.7^\circ\text{C}$), preserving downstream rolling statistics without discontinuity. Similarly, the missing CO₂ reading at 10:04 was smoothly interpolated to 735.0 ppm.

Compound Multi-Sensor Incident Detection: The system excelled at capturing overlapping failures. Between 10:32 and 10:35, the classroom reached an acute risk state where poor indoor air quality, thermal discomfort, and heavy workstation electrical demand converged simultaneously.

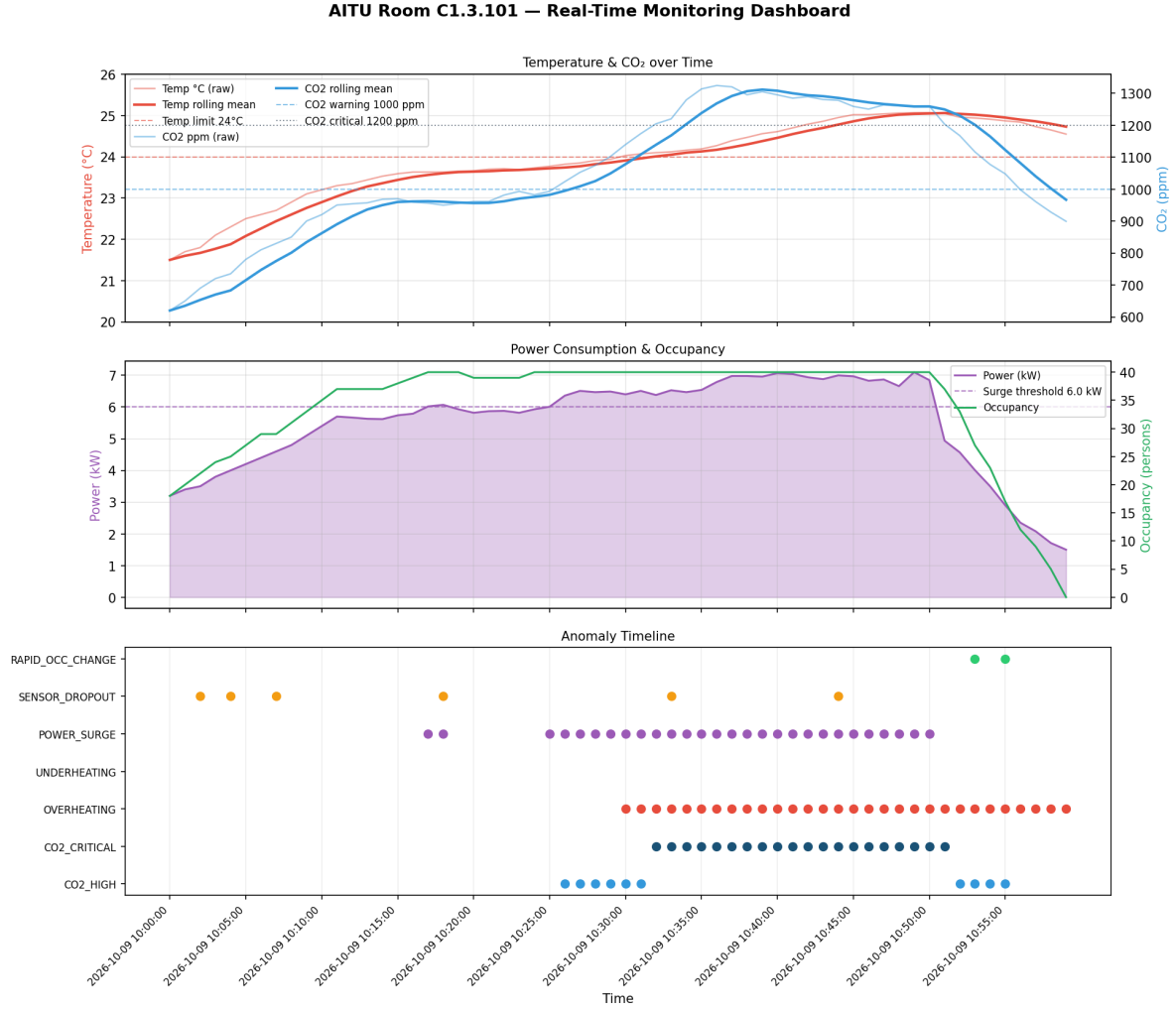


Figure 1: AITU Room C1.3.101 – Real-Time Stream Monitoring and Anomaly Dashboard (3-panel output generated by `Midterm_Project.ipynb`)

5.2 Limitations and Future Work

While the in-memory simulation accurately models sensor dynamics, industrial deployment would incorporate:

- **Message Broker Integration:** Connecting to Apache Kafka or MQTT brokers to ingest real physical IoT streams.
- **Adaptive Dynamic Baselines:** Replacing static thresholds with rolling z -scores ($\mu \pm 3\sigma$) or contextual thresholds conditioned on class schedule and external weather.
- **Automated Actuation:** Feeding alerts into Building Management Systems (BMS) to automatically cycle HVAC fans when CO₂ exceeds 1000 ppm.

6 Conclusion

This midterm project successfully designed and implemented an end-to-end real-time stream processing system for classroom monitoring. By taking raw sensor records one-by-one, handling dropouts via online extrapolation, calculating sliding-window indicators,

and triggering multi-tier anomaly alarms, the architecture satisfies all requirements of modern stream computing.

The project underscores key differences between streaming and batch paradigms: real-time processing mandates strict state management, ring buffer discipline, and causal time guarantees.